

MPOO · GRUPO 6 · S14 · 2026-10-05

Repaso de la Unidad 1

Repasemos lo que ya hicimos: P3, P4, P5 y T01 a T03.

100 minutos útiles · Java 21 · FI-UNAM · 2027-1

Predice primero. Comprueba después. Explica con tus palabras.

LO QUE REPASAMOS HOY

La Unidad 1, con lo que ya hicimos

Tema	De dónde lo retomamos
Conceptos clave	Clases y objetos, métodos, variables, tipos y referencias
Leer código y predecir la salida	P3, S07 y S09
Compila, trueno o funciona	P4, P5 y S12
Modelado de clases	T01 y P5
Estructuras de repetición	S12, P4 y T03
Estructuras de selección	T02 y P5

Predice primero, comprueba después y explica con tus palabras.

CONCEPTOS DE LA U1

Conceptos clave: explícalos con tus palabras

Tema	Preguntas para explicar
Objetos y clases	Clase frente a objeto; qué es una instancia; qué hace new.
Métodos	Qué es la firma; qué hace main y por qué es static; constructor frente a método.
Variables y ámbito	Anatomía de una variable local frente a un atributo; static frente a instancia; this.
Tipos y referencias	Primitivo frente a referencia; equals frente a ==; null; paso por valor.
Control y la máquina	while frente a do-while; break en switch; javac y la JVM; JDK y JRE.

Si puedes explicarlo en una a tres oraciones, con un ejemplo pequeño, lo entendiste.

 Tu turno · 4 min

Tu turno 0 · contesta con tus palabras

- ◆ ¿Qué es un wrapper como Integer y qué es el autoboxing?
- ◆ ¿Para qué sirve el parámetro `String[] args` del main?
- ◆ ¿Qué hace el operador ternario y cuándo conviene usarlo en lugar de un if?

Escríbelo antes de ver la respuesta. Importa la idea, no la redacción perfecta.

Lo que no puede faltar

Pregunta	Idea indispensable
Wrapper y autoboxing	Integer envuelve un int en un objeto; el autoboxing convierte int en Integer (y de regreso) sin escribirlo.
String[] args	Es un arreglo con los textos que se escriben al ejecutar el programa; revisa su length antes de leerlo.
Ternario	condición ? valorSiVerdadero : valorSiFalso devuelve un valor; conviene para elegir entre dos valores en una línea.

Usa el vocabulario de clase: plano, instancia, firma, anatomía, ámbito.

SECCIÓN 1

Leer código: ¿qué imprime?

🕒 ~14 min

Lee la consigna, deja una respuesta y justifica cómo llegaste.

P3 · BLOQUES 2 Y 4

División entera y casting

```
System.out.println(7 / 2);  
System.out.println(7.0 / 2);  
System.out.println((double) 7 / 2);  
System.out.println((double) (7 / 2));  
System.out.println((int) -3.9);
```

```
3  
3.5  
3.5  
3.0  
-3
```

Convertir después de dividir no recupera la fracción. El cast a int trunca hacia cero.

P3 · BLOQUE 6

Suma y concatenación

```
System.out.println(5 + 3);  
System.out.println("5" + 3);  
System.out.println(1 + 2 + "3");  
System.out.println("1" + 2 + 3);
```

```
8  
53  
33  
123
```

El + se evalúa de izquierda a derecha: con texto concatena; con números suma.

== frente a equals

```
String a = "Hola";  
String b = "Hola";  
String c = new String("Hola");  
System.out.println(a == b);  
System.out.println(a == c);  
System.out.println(a.equals(c));  
String s = "hola";  
s.toUpperCase();  
System.out.println(s);
```

```
true  
false  
true  
hola
```

En String, equals compara contenido. toUpperCase devuelve un resultado que aquí no guardamos.

S09 · ARREGLOS

Dos referencias al mismo arreglo

```
int[] notas = {8, 9, 10};  
int[] copia = notas;  
copia[0] = 5;  
System.out.println(notas[0]);
```

5

Asignar notas a copia no crea otro arreglo. En P4, invertido debe devolver uno nuevo.

 Tu turno · 3 min

Tu turno 1 · suma de la P4

```
static int suma(int[] numeros) {  
    int total = 0;  
    for (int i = 0; i < numeros.length; i++) {  
        total = total + numeros[i];  
    }  
    return total;  
}
```

En el main: notas = {7, 9, 5, 10, 8}. ¿Qué devuelve suma(notas)?

Anota i , $\text{numeros}[i]$ y total en cada vuelta. Es el ejemplo A de P4Arreglos.

TU TURNO 1 · SOLUCIÓN

Recorrido de suma(notas)

```
static int suma(int[] numeros) {  
    int total = 0;  
    for (int i = 0; i < numeros.length; i++) {  
        total = total + numeros[i];  
    }  
    return total;  
}
```

i	numeros[i]	total después
0	7	7
1	9	16
2	5	21
3	10	31
4	8	39

Empieza en 0; suma los cinco valores y retorna 39. No imprime dentro del método.

TU TURNO 1 · SALIDA Y CASO VACÍO

Retorno e impresión

```
int[] notas = {7, 9, 5, 10, 8};  
System.out.println(suma(notas));  
System.out.println(suma(new int[]{}));
```

```
39  
0
```

El main imprime el valor retornado. Con un arreglo vacío, el ciclo no entra y devuelve 0.

 Tu turno · 3 min

Tu turno 2 · Predicciones de la P3

```
int vidasRestantes = 0;  
System.out.println(12345 % 10);  
System.out.println(12345 / 10);  
System.out.println((char) ('A' + 1));  
System.out.println(vidasRestantes > 0 && 10 / vidasRestantes > 1);
```

Escribe las cuatro líneas. En la última, decide si Java llega a dividir entre cero.

TU TURNO 2 · SOLUCIÓN

Residuo, división, char y cortocircuito

```
5  
1234  
B  
false
```

Expresión	Razón
12345 % 10	El residuo es el último dígito: 5.
12345 / 10	La división entera deja 1234.
(char) ('A' + 1)	65 + 1 se convierte al carácter B.
vidasRestantes > 0 && ...	La izquierda es falsa: no se evalúa la división.

Las cuatro expresiones vienen de los bloques 3, 4 y 5 de Laboratorio.java.

SECCIÓN 2

¿Compila, truena o funciona?

🕒 ~15 min

Lee la consigna, deja una respuesta y justifica cómo llegaste.

Compilación y ejecución

No compila: tipo o ámbito

```
int piezas = 4.6;
```

```
for (int i = 0; i < 3; i++) { }  
System.out.println(i);
```

Compila, pero truena: índice inválido

```
int[] numeros = {10, 20, 30};  
System.out.println(numeros[numeros.length]);
```

javac comprueba tipos y ámbito; el acceso fuera del arreglo falla cuando la JVM lo ejecuta.

Compila, pero no termina

```
int i = 0;
while (i < 5) {
    // Falta i++; no hay avance.
}
```

Compilar no demuestra que el programa resuelva el problema.

- ◆ i sigue valiendo 0.
- ◆ La condición sigue siendo verdadera.
- ◆ No hay una excepción: hay una repetición sin salida.

Tu turno 3 · La Expendedora de la P5

a) Con las clases resueltas de P5

```
Expendedora e = new Expendedora("Edificio Q", 2);  
e.papas = null; // Simula que faltó el new de Producto.  
System.out.println(e.papas.hayPiezas());
```

b) Con buscar resuelto

```
Expendedora e = new Expendedora("Edificio Q", 2);  
Producto p = e.buscar(" a2 ");  
System.out.println(p.getPrecio());
```

c) Independiente

```
Producto.CAPACIDAD = 12;
```

Clasifica cada inciso y justifica. Si funciona, escribe la salida.

Null, búsqueda y constante

Inciso	Resultado	Causa
a	Compila, truená: NullPointerException	papas vale null; no se puede invocar hayPiezas().
b	Funciona: imprime 22	buscar limpia espacios y mayúsculas: " a2 " se vuelve "A2".
c	No compila	CAPACIDAD ya tiene valor y es final.

Declarar la referencia no crea el Producto. El new debe ocurrir en el constructor.

Olvidar this y cambiar el orden

```
class Expendedora {  
    String ubicacion;  
    Expendedora(String ubicacion) {  
        ubicacion = ubicacion;  
    }  
}
```

- ◆ Sin this, el atributo ubicacion sigue en null.
- ◆ Sin trim().toUpperCase(), buscar no reconoce " a2 ".
- ◆ En comprar: código inválido, piezas, saldo; en ese orden.

Estos errores pueden compilar. Una prueba debe comprobar el resultado, no solo que arranque.

Wrappers y StringBuilder

```
Integer n = 42;  
int m = n;  
System.out.println(m);  
StringBuilder sb = new StringBuilder();  
for (int i = 0; i < 3; i++) {  
    sb.append(i).append(", ");  
}  
System.out.println(sb.toString());  
System.out.println(Math.abs(0.1 + 0.2 - 0.3) < 0.000001);
```

```
42  
0, 1, 2,  
true
```

Integer envuelve int; sb acumula texto. args también es un arreglo: revisa su longitud antes de acceder.

SECCIÓN 3

Modelado de clases

🕒 ~15 min

Lee la consigna, deja una respuesta y justifica cómo llegaste.

Del contexto a la clase

Pregunta	Qué escribes
¿En qué programa vive?	Contexto y un detalle que ignoras.
¿Qué necesita recordar?	Atributos con tipo y razón.
¿Cómo nace?	Constructor: parámetros, this y estado inicial.
¿Qué hace o responde?	Cabeceras con retorno, nombre y parámetros; una razón.

Sin diagrama. La firma Java estricta es el nombre y los tipos de parámetros.

Producto: atributos y métodos

```
class Producto {  
    static final int CAPACIDAD = 10;  
    String nombre;  
    int precio;  
    int piezas;  
    int getPrecio() { return precio; }  
    int getPiezas() { return piezas; }  
    boolean hayPiezas() { return piezas > 0; }
```

Dato	Por qué
nombre	Texto para identificar el producto.
precio / piezas	Precio entero y existencia actual.
CAPACIDAD	static final: máximo común de 10, no reasignable.

Producto recuerda su propia existencia; no necesita guardar el saldo de la máquina.

Constructor de Producto

```
Producto(String nombre, int precio, int piezas) {  
    this.nombre = nombre;  
    this.precio = precio;  
    if (piezas < 0) {  
        this.piezas = 0;  
    } else if (piezas > CAPACIDAD) {  
        this.piezas = CAPACIDAD;  
    } else {  
        this.piezas = piezas;  
    }  
}
```

piezas recibidas	piezas guardadas
-1	0
15	10
1	1

this.piezas es el atributo. El parámetro puede estar fuera del rango y se ajusta al construir.

Expendedora: qué guarda y por qué

```
class Expendedora {  
    static int ventasTotales = 0;  
    String ubicacion;  
    int saldo = 0;  
    Producto papas;  
    Producto refresco;  
    Producto chocolate;  
}
```

Dato	Responsabilidad
ubicacion	Identifica dónde está esta máquina.
saldo	Dinero insertado en esta máquina.
papas, refresco, chocolate	Referencias a sus tres Producto.
ventasTotales	static: ventas de todas las máquinas.

No todo es int: los tres productos son objetos con sus propios datos y operaciones.

Constructor de Expendedora

```
Expendedora(String ubicacion, int piezas) {  
    this.ubicacion = ubicacion.isBlank() ? "SIN UBICACION" : ubicacion;  
    this.papas = new Producto("papas", 18, piezas);  
    this.refresco = new Producto("refresco", 22, piezas);  
    this.chocolate = new Producto("chocolate", 15, piezas);  
}
```

Saldo empieza en 0. Cada producto recibe sus piezas; Producto valida su capacidad.

Tres new crean tres productos. El ternario conserva una ubicación o usa SIN UBICACION.

Métodos de Expendedora

Cabecera	Para qué existe
Producto buscar(String codigo)	Devuelve el producto o null.
void insertar(Moneda moneda)	Cambia saldo; no devuelve un valor.
String comprar(String codigo)	Intenta vender y devuelve el resultado.
int cancelar()	Devuelve saldo y lo deja en 0.
static int getVentasTotales()	Consulta el contador compartido.

Una cabecera comunica qué recibe y devuelve. Elige static solo si la operación es de la clase.

 Tu turno · 4 min

Tu turno 4 · Impresora de la T01

Retoma la Impresora del menú de ideas. Guarda `hojas`, `copiasHechas` y `tinta`. El constructor recibe `hojas` y `tinta`; `copiasHechas` inicia en 0. `imprimir(int n)` descuenta hojas y suma copias si `n` es positivo y alcanza el papel. `tieneHojas()` responde si queda papel.

Escribe atributos, constructor y cabeceras. Justifica tipos y qué dejarías fuera.

Contexto: controlar las copias. No necesitas modelar marca, peso ni color de la carcasa.

Atributos y constructor de Impresora

```
class Impresora {  
    int hojas;  
    int copiasHechas = 0;  
    char tinta;  
    Impresora(int hojas, char tinta) {  
        this.hojas = hojas;  
        this.tinta = tinta;  
    }  
}
```

Elección	Razón
int hojas	Cantidad de papel disponible.
int copiasHechas	Trabajo acumulado de esta impresora.
char tinta	Código de tinta, como en la T01.
Sin static ni final	Son datos de cada objeto que pueden cambiar.

El constructor no lleva retorno. this distingue atributos de parámetros.

TU TURNO 4 · OPERACIONES POSIBLES

Imprimir y consultar papel

```
void imprimir(int n) {  
    if (n > 0 && n <= hojas) {  
        hojas -= n;  
        copiasHechas += n;  
    }  
}  
  
boolean tieneHojas() { return hojas > 0; }
```

Cabecera	Razón
void imprimir(int n)	Recibe cuántas copias; modifica el estado.
boolean tieneHojas()	Consulta sin parámetros; devuelve sí o no.

El cuerpo es una solución posible; en modelado justificas también por qué existe cada método.

SECCIÓN 4

Estructuras de repetición

🕒 ~18 min

Lee la consigna, deja una respuesta y justifica cómo llegaste.

P4, S12 Y T03

Elegir el ciclo

Ciclo	Ejercicio conocido	Condición
for	suma de P4 o factorial de T03	Índice o contador antes de cada vuelta.
for-each	sumaPares de S12	Quedan valores por recorrer.
while	Contraseña de T03	Quedan intentos y no acertó.
do-while	contarDigitos o menú de T03	Se pregunta después del cuerpo.

Elige según el recorrido. En todos, identifica qué hace que termine.

Límite del arreglo

```
int[] numeros = {10, 20, 30};  
for (int i = 0; i <= numeros.length; i++) {  
    System.out.println(numeros[i]);  
}
```

```
int[] numeros = {10, 20, 30};  
for (int i = 0; i < numeros.length; i++) {  
    System.out.println(numeros[i]);  
}
```

Con <= imprime 10, 20 y 30, y luego truena. length no es un índice válido.

Valor inicial del máximo

```
int[] numeros = {-5, -2, -9};  
int max = 0;  
for (int i = 0; i < numeros.length; i++) {  
    if (numeros[i] > max) { max = numeros[i]; }  
}  
System.out.println(max);
```

```
int[] numeros = {-5, -2, -9};  
int max = numeros[0];  
for (int i = 1; i < numeros.length; i++) {  
    if (numeros[i] > max) { max = numeros[i]; }  
}  
System.out.println(max);
```

Con {-5, -2, -9}, 0 es una respuesta falsa; el máximo es -2. El arreglo debe tener elementos.

S12 · ERROR 3

Actualización del ciclo

```
int i = 0;
while (i < 5) {
    // Falta i++; no hay avance.
}
```

```
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++;
}
```

Con `i++` se imprimen 0, 1, 2, 3 y 4. Sin avance, `i` nunca deja de ser 0.

Contar dígitos: prueba de escritorio

n antes de dividir	n / 10	Dígitos contados
2026	202	1
202	20	2
20	2	3
2	0	4

Los tres casos de clase: contarDigitos(7) da 1; contarDigitos(2026), 4; contarDigitos(0), 1.

Con 0 hay que contar un dígito. Recuerda por qué en la S12 usamos do-while para ese caso.

T03 · EJEMPLO RESUELTO A MANO

Factorial: contador y acumulador

```
// La calculadora ya validó 0 a 20.  
int n = 5;  
long factorial = 1;  
for (int i = 2; i <= n; i++) {  
    factorial = factorial * i;  
}  
System.out.println(factorial);
```

Es el case 5 de Calculadora.java. factorial inicia en 1; i recorre de 2 a n.

Factorial: multiplicar en cada vuelta

```
// La calculadora ya validó 0 a 20.  
int n = 5;  
long factorial = 1;  
for (int i = 2; i <= n; i++) {  
    factorial = factorial * i;  
}  
System.out.println(factorial);
```

Con $n = 5$, factorial pasa por 2, 6, 24 y 120. Con $n = 0$ permanece en 1.

 Tu turno · 4 min

Tu turno 5 · Contraseña de la S12 y T03

Vuelve a escribir la contraseña, ahora con `do-while`. Usa Scanner, `secreta = "mpoo2027"`, `intentos`, `correcta` e `intento`. Limpia espacios con `trim` y compara con `equals`. Máximo tres intentos. Tras un error avisa cuántos quedan; al final, Bienvenido o Cuenta bloqueada.

El primer intento siempre ocurre. Después pregunta: ¿quedan intentos y sigue incorrecta?

Contraseña con do-while

```
Scanner sc = new Scanner(System.in);
String secreta = "mpoo2027";
int intentos = 0;
boolean correcta = false;
do {
    String intento = sc.nextLine().trim();
    intentos++;
    correcta = intento.equals(secreta);
    if (!correcta && intentos < 3) {
        System.out.println("Quedan " + (3 - intentos));
    }
} while (intentos < 3 && !correcta);
System.out.println(correcta ? "Bienvenido" : "Cuenta bloqueada");
```

intentos aumenta siempre. La condición lleva &&: basta acertar o gastar tres intentos para salir.

Dos formas de salir

Entradas	intentos al salir	correcta	Mensaje final
mpoo2027	1	true	Bienvenido
hola, mpoo2027	2	true	Bienvenido
a, b, mpoo2027	3	true	Bienvenido
a, b, c	3	false	Cuenta bloqueada

El tercer intento todavía puede acertar. Los espacios se limpian antes de comparar.

Sumar pares: condición y acumulador

Valor x de numeros	¿x % 2 == 0?	suma después
3	no	0
8	sí	8
5	no	8
12	sí	20
7	no	20

En la S12 sumamos $8 + 12 = 20$ con for-each. suma empieza en 0; solo cambia al encontrar un par.

 Tu turno · 4 min

Tu turno 6 · sumaDeFila de la P4

Completa `static int sumaDeFila(int[][] matriz, int fila)`, el ejercicio 7 de P4Arreglos. Suma los valores de la fila indicada usando `for-each`. La matriz y esa fila no son null; el índice `fila` es válido.

matriz	fila	Debe devolver
{{1, 2, 3}, {10, 20, 30}}	1	60
La misma matriz	0	6

`matriz[fila]` es un arreglo. Recorre sus valores; no hace falta recorrer todas las filas.

TU TURNO 6 · SOLUCIÓN

Sumar una fila con for-each

```
static int sumaDeFila(int[][] matriz, int fila) {  
    int total = 0;  
    for (int valor : matriz[fila]) {  
        total += valor;  
    }  
    return total;  
}
```

valor de matriz[1]	total
10	10
20	30
30	60

El método devuelve 60 para la fila 1 de P4. Si la fila está vacía, devuelve 0.

El menú se repite hasta elegir 0

```
Scanner sc = new Scanner(System.in);
int opcion;
do {
    System.out.println("1 Sumar | 2 Restar | 3 Multiplicar");
    System.out.println("4 Dividir | 5 Factorial | 0 Salir");
    opcion = sc.nextInt();
    // Aquí va el switch de Calculadora.java.
} while (opcion != 0);
System.out.println("Adios");
```

Este extracto muestra solo el ciclo del menú. El switch con las operaciones va dentro del do.

Factorial con for

```
// La calculadora ya validó 0 a 20.  
int n = 5;  
long factorial = 1;  
for (int i = 2; i <= n; i++) {  
    factorial = factorial * i;  
}  
System.out.println(factorial);
```

i	factorial
Inicio	1
2	2
3	6
4	24
5	120

Primero valida 0 a 20. Con $n = 0$ no hay vueltas: $0!$ vale 1. La calculadora usa long.

SECCIÓN 5

Estructuras de selección

🕒 ~20 min

Lee la consigna, deja una respuesta y justifica cómo llegaste.

Bisiesto: condiciones combinadas

```
static boolean esBisiesto(int anio) {  
    return anio % 4 == 0 &&  
        (anio % 100 != 0 || anio % 400 == 0);  
}
```

anio	esBisiesto
2024	true
1900	false
2000	true
2026	false

El switch sobre anio no expresa divisibilidad con los case constantes vistos. Aquí usamos %, && y ||.

P5 · VARIANTE DE BUSCAR PARA OBSERVAR LA CAÍDA

Switch clásico sin break

```
String codigo = "A1";  
switch (codigo) {  
    case "A1": System.out.println("papas");  
    case "A2": System.out.println("refresco"); break;  
    case "B1": System.out.println("chocolate"); break;  
    default: System.out.println("CODIGO INVALIDO");  
}
```

```
papas  
refresco
```

Entra en A1 y continúa en A2. En buscar, conserva los break para devolver el Producto correcto.

Switch moderno sobre enum Moneda

```
void insertar(Moneda moneda) {  
    switch (moneda) {  
        case UNO -> saldo += 1;  
        case DOS -> saldo += 2;  
        case CINCO -> saldo += 5;  
        case DIEZ -> saldo += 10;  
    }  
}
```

Moneda declara UNO, DOS, CINCO y DIEZ; el saldo final es 18.

```
Expendedora e = new Expendedora("Edificio Q", 2);  
e.insertar(Moneda.UNO);  
e.insertar(Moneda.DOS);  
e.insertar(Moneda.CINCO);  
e.insertar(Moneda.DIEZ);  
System.out.println(e.saldo);
```

Las flechas no continúan al caso siguiente. insertar cambia saldo y devuelve void.

T02 · EJEMPLO RESUELTO

Calificaciones con if y else-if

```
static String letraConIf(int calificacion) {  
    if (calificacion >= 90) { return "A"; }  
    else if (calificacion >= 80) { return "B"; }  
    else if (calificacion >= 70) { return "C"; }  
    else if (calificacion >= 60) { return "D"; }  
    else { return "F"; }  
}
```

calificacion	Letra
90 a 100	A
80 a 89	B
70 a 79	C
60 a 69	D
0 a 59	F

Se recibe de 0 a 100, como en la T02. La primera condición verdadera decide la letra.

 Tu turno · 5 min

Tu turno 7 · Calificaciones con switch

Reescribe `static String letraConSwitch(int calificacion)` de la T02. Recibe de 0 a 100 y devuelve la misma letra que `letraConIf`. Usa `switch` sobre `calificacion / 10`. Puedes usar la forma clásica o moderna.

Entradas para revisar	Resultados
59 / 60 / 89	F / D / B
90 / 100	A / A

Revisa 100: al dividir entre 10 produce un caso distinto de 90.

TU TURNO 7 · SOLUCIÓN CLÁSICA

Agrupar 9 y 10 en la letra A

```
static String letraConSwitch(int calificacion) {  
    switch (calificacion / 10) {  
        case 10:  
        case 9: return "A";  
        case 8: return "B";  
        case 7: return "C";  
        case 6: return "D";  
        default: return "F";  
    }  
}
```

return sale del método; aquí no hace falta break. default cubre 0 a 59 dentro del rango dado.

Java copia el valor del argumento

```
static void subir(int n) { n = n + 10; }  
static void subir(Coche c) { c.anio = 2020; }
```

```
int edad = 20;  
subir(edad);  
System.out.println(edad);  
Coche mio = new Coche("Tsuru", 1998);  
subir(mio);  
System.out.println(mio.anio);
```

```
20  
2020
```

subir(edad) cambia una copia del entero. subir(mio) recibe una copia de la referencia al mismo Coche.

PARA PRACTICAR

Antes de la siguiente clase

- ◆ Vuelve a resolver T01, T02, T03 y P3 a P5 sin mirar sus soluciones.
- ◆ Haz prueba de escritorio: un caso normal y uno borde (cero, igualdad, arreglo corto).
- ◆ Explica en voz alta cada concepto clave con un ejemplo de código.

Si te atorras, escribe la estructura que sí sabes y luego completa el detalle.